Análisis en Lean 4

Alex Kontorovich

Traducido por: José A. Alonso

Sevilla, 1 de octubre de 2026

Índice general

1. La historia del análisis real **20**

1.1. Preámbulo 20

1.2. Introducción a Lean 27

1.3. La táctica `rfl` 28

1.4. La táctica `rw` 29

1.5. La táctica `ring_nf` 30

1.6. La táctica `use` 31

1.7. La táctica `intro` 31

1.8. La táctica `specialize` 32

1.9. El desafío táctico definitivo 35

2. El cálculo de Newton del número pi **38**

2.1. La revolución matemática de 1666 38

2.2. La definición principal 52

3. Convergencia de sucesiones **64**

3.1. Más sobre sucesiones 64

3.2. Nivel 1: La propiedad arquimediana 64

3.2.1. La estrategia de demostración en lenguaje natural 65

3.2.2. Los desafíos de la implementación en Lean 66

3.2.3. Nuevas herramientas que necesitarás 68

3.2.4. Demostraciones en Lean 68

3.2.5. Demostración en lenguaje natural de la propiedad arquimediana . 71

3.2.6. Revisión de errores comunes 72

3.3. Nivel 2: Nuestro primer límite real 73

3.3.1. El objetivo: Demostrar que $1/n \to 0$ 73

3.3.2. Recordatorio: La definición de convergencia de sucesiones 73
3.3.3. Estrategia de demostración en lenguaje natural 74
3.3.4. Los desafíos de la implementación en Lean 75
3.3.4.1. Desafío 1: Multiplicación cruzada de fracciones 75
3.3.4.2. Desafío 2: Aritmética lineal 76
3.3.4.3. Desafío 3: Casting explícito de tipos 76
3.3.4.4. Desafío 4: Casting en aplicaciones de tácticas 76
3.3.5. Nuevas herramientas que necesitarás 77
3.3.6. Consejos profesionales para este nivel 77
3.3.7. Demostración en Lean 78
3.3.8. Lo que acabamos de lograr 82

4. No convergencia **84**
4.1. Un gran debate 84
4.2. Paso 2: Solución en Lean 89
4.3. Paso 3: Demostración en lenguaje natural 96
4.3.1. Demostración en lenguaje natural 96
4.3.2. Ideas clave 97

5. Duplicar una sucesión convergente **98**
5.1. El desafío de escalar en la fábrica 98
5.2. La configuración matemática 99
5.2.1. Idea Clave: Escalado Inverso de la Tolerancia 99
5.3. Nuevas herramientas que necesitarás 100
5.3.1. Valor absoluto de productos 100
5.4. Enfoque estratégico 100
5.5. Solución en Lean 101
5.6. Prueba en lenguaje natural 109
5.6.1. Enunciado del teorema 109
5.6.2. Prueba 109
5.7. Por qué importa este patrón 110
5.7.1. La estrategia elegante 111

6. Técnicas avanzadas de demostración y el teorema del emparedado **113**

6.1. La suma de sucesiones convergentes . . . 113
6.1.1. La configuración matemática . . . 113
6.1.2. Estrategia . . . 114
6.1.3. Solución en Lean . . . 114
6.1.4. Demostración en Lenguaje Natural . . . 123
6.2. Separar conjunciones: Descomponiendo metas complejas . . . 123
6.2.1. Nuevas herramientas . . . 123
6.2.2. Ejemplo . . . 124
6.3. Izquierda y derecha: Tomando decisiones en matemáticas . . . 125
6.3.1. Nuevas herramientas . . . 125
6.3.2. Ejemplo . . . 126
6.4. Notación de punto: Accediendo a partes de información compleja . . . 127
6.4.1. Reglas de la notación de punto . . . 127
6.4.2. Ejemplo . . . 127
6.5. `cases'`: Manejando todas las posibilidades . . . 128
6.5.1. La Táctica `cases'` . . . 128
6.5.2. Ejemplo . . . 129
6.5.3. El conjunto completo de herramientas y/o . . . 130
6.6. `abs_lt`: Trabajando con valores absolutos en convergencia . . . 130
6.6.1. El teorema `abs_lt` . . . 131
6.6.2. Ejemplo de aplicación . . . 131
6.7. El teorema del emparedado . . . 135
6.7.1. La imagen intuitiva . . . 135
6.7.2. El enunciado matemático . . . 135
6.7.3. Demostración en Lean . . . 135
6.7.4. Demostración en lenguaje natural . . . 142
6.7.5. Aplicaciones . . . 142
6.8. Ejercicios . . . 143

7. Unicidad y teoremas avanzados de límites 155
7.1. Unicidad de límites . . . 155
7.1.1. Nuevas herramientas . . . 155
7.1.1.1. Prueba por contradicción: `by_contra` . . . 155

7.1.1.2. Positividad del valor absoluto: abs_pos_of_nonzero . . . 155
7.1.2. Enunciado matemático . . . 156
7.1.3. Demostración en Lean . . . 156
7.1.4. Prueba en lenguaje natural . . . 160
7.1.5. Por qué importa la unicidad . . . 160
7.2. Las sucesiones convergentes a un límite no nulo, eventualmente se alejan de cero . . . 160
7.2.1. Enunciado matemático . . . 161
7.2.2. Enfoque estratégico . . . 161
7.2.3. Demostración en Lean . . . 161
7.2.4. Prueba en lenguaje natural . . . 164
7.2.5. Aplicaciones . . . 164
7.3. Continuidad del valor absoluto: Sucesiones de valores absolutos . . . 164
7.3.1. Nuevas herramientas . . . 164
7.3.1.1. Propiedad de Lipschitz: abs_Lipschitz . . . 164
7.3.2. Enunciado matemático . . . 165
7.3.3. Demostración en Lean . . . 165
7.3.4. Prueba en lenguaje natural . . . 169
7.3.5. Por qué importa este resultado . . . 169
7.4. Recíprocos de sucesiones convergentes . . . 169
7.4.1. Nuevas herramientas . . . 169
7.4.1.1. Valor absoluto de cocientes: abs_div . . . 169
7.4.1.2. No nulo a partir de valor absoluto positivo: nonzero_of_abs_pos 169
7.4.2. Enunciado matemático . . . 170
7.4.3. Enfoque estratégico . . . 170
7.4.4. Demostración en Lean . . . 170
7.4.5. Prueba en lenguaje natural . . . 178
7.4.6. Prueba conseguida . . . 178
7.4.7. Lo que lograste . . . 178
7.4.8. Aplicaciones y extensiones . . . 179
7.4.9. Dominio de la técnica . . . 179
7.5. Ejercicios . . . 179

8. Teoremas avanzados de límites e inducción 191

8.1. Introducción: Inducción matemática . . . 191
8.2. Juego final: Recíprocos de sucesiones convergentes . . . 193
8.2.1. El Marco Matemático . . . 194
8.2.2. Nuevas Herramientas . . . 194
8.2.3. Enfoque estratégico . . . 194
8.2.4. Solución en Lean . . . 194
8.2.5. Demostración en lenguaje natural . . . 202
8.2.6. Aplicaciones y extensiones . . . 203
8.3. Por casos: Análisis de casos sin hipótesis . . . 203
8.3.1. La táctica `by_cases` . . . 203
8.3.2. Aplicación: Eventualmente mayor que la mitad del límite . . . 203
8.3.3. Solución en Lean . . . 204
8.3.4. Demostración en lenguaje natural . . . 208
8.3.5. Importancia . . . 208
8.4. Inducción matemática . . . 208
8.4.1. El principio de inducción matemática . . . 209
8.4.2. La Táctica `induction'` . . . 209
8.4.3. Nueva Herramienta: `ge_one_of_nonzero` . . . 209
8.4.4. Ejemplo: Crecimiento exponencial . . . 209
8.4.5. Solución en Lean . . . 209
8.4.6. Demostración en Lenguaje Natural . . . 213
8.4.7. Aplicaciones . . . 213
8.4.8. Idea clave . . . 214
8.5. Ejercicios . . . 214

9. Acotación de sucesiones convergentes 231
9.1. Sumas finitas: cada término acotado por el total . . . 231
9.1.1. Nuevas herramientas . . . 231
9.1.1.1. Notación de suma: `range` . . . 231
9.1.1.2. Sucesor del rango de suma: `sum_range_succ` . . . 231
9.1.1.3. No negatividad de sumas: `sum_nonneg` . . . 232
9.1.1.4. Prueba por contradicción: `contradiction` . . . 232
9.1.2. El enunciado matemático . . . 232

9.1.3. Enfoque estratégico . . . 232
9.1.4. Solución en Lean . . . 232
9.1.5. Demostración en lenguaje natural . . . 237
9.1.6. Por qué importa este resultado . . . 238
9.2. Acotamiento de sucesiones convergentes . . . 238
9.2.1. Nuevas herramientas . . . 239
9.2.1.1. Definición: `SeqBdd` . . . 239
9.2.2. El enunciado matemático . . . 239
9.2.3. Enfoque estratégico . . . 239
9.2.4. Solución en Lean . . . 239
9.2.5. Demostración en Lenguaje Natural . . . 240
9.2.6. Importancia del Acotamiento . . . 240

10. Álgebra de límites y propiedades de orden 243
10.1. Producto de sucesiones . . . 243
10.1.1. El enunciado matemático . . . 243
10.1.2. Enfoque estratégico . . . 243
10.1.3. Demostración en Lean . . . 244
10.1.4. Demostración en lenguaje natural . . . 248
10.1.5. Conexión con el cálculo: ¿por qué sumar y restar $b(n) \cdot L$? . . . 249
10.1.6. Completando el teorema del límite algebraico . . . 249
10.2. Paso al límite en desigualdades . . . 250
10.2.1. Nuevas herramientas . . . 250
10.2.2. El enunciado matemático . . . 250
10.2.3. Enfoque estratégico . . . 250
10.2.4. Demostración en Lean . . . 250
10.2.5. Demostración en lenguaje natural . . . 254
10.2.6. Importancia de la preservación del orden . . . 255
10.3. Subsucesiones . . . 255
10.3.1. Nuevas herramientas . . . 255
10.3.2. El enunciado matemático . . . 256
10.3.3. Enfoque estratégico . . . 256
10.3.4. Demostración en Lean . . . 256

10.3.5. Demostración en lenguaje natural . . . 256
10.3.6. Por qué importa . . . 257
10.4. Ejemplo de subsucesión: sucesiones oscilantes . . . 257
10.4.1. Nuevas herramientas . . . 257
10.4.2. El enunciado matemático . . . 258
10.4.3. Enfoque estratégico . . . 258
10.4.4. Demostración en Lean . . . 258
10.4.5. Demostración en lenguaje natural . . . 258
10.4.6. Implicaciones para la divergencia . . . 259

11. Los números reales I **188**
11.1. Introducción: Sucesiones de Cauchy y los números reales . . . 188
11.2. Nivel superior: Los límites son Cauchy . . . 193
11.2.1. Planteamiento matemático . . . 193
11.2.2. Nuevas herramientas . . . 193
11.2.3. Enfoque estratégico . . . 193
11.2.4. Solución en Lean . . . 194
11.2.5. Demostración en lenguaje natural . . . 195
11.3. Nivel 2: Sumas de sucesiones de Cauchy . . . 195
11.3.1. Planteamiento matemático . . . 196
11.3.2. Enfoque estratégico . . . 196
11.3.3. Solución en Lean . . . 196
11.3.4. Demostración en lenguaje natural . . . 197
11.4. Nivel 3: Cauchy implica acotada . . . 198
11.4.1. Planteamiento matemático . . . 198
11.4.2. Enfoque estratégico . . . 198
11.4.3. Solución en Lean . . . 199
11.4.4. Demostración en lenguaje natural . . . 200
11.4.5. Importancia . . . 200

12. Las sucesiones monótonas acotadas son de Cauchy **202**
12.1. Subsucesiones iteradas y órbitas . . . 202
12.1.1. La idea clave: órbitas . . . 203
12.1.2. Aplicación a nuestro problema . . . 203

12.1.3. Nuevas herramientas . . . 204
12.1.3.1. Iteración de funciones: `succ_iterate` . . . 204
12.1.3.2. Subsucesión a partir del sucesor: `subseq_of_succ` . . . 204
12.1.3.3. Táctica: `show` . . . 205
12.1.4. El enunciado matemático . . . 205
12.1.5. Estrategia de la demostración . . . 205
12.1.6. Demostración en Lean . . . 206
12.1.7. Demostración en lenguaje natural . . . 206
12.1.8. Por qué esto importa . . . 207
12.2. Elección mejorada: de la existencia a la estructura . . . 207
12.2.1. Interpretar la existencia como funciones . . . 208
12.2.2. De la existencia a las subsucesiones . . . 209
12.2.3. El enunciado matemático . . . 210
12.2.4. Estrategia de la demostración . . . 210
12.2.5. Demostración en Lean . . . 211
12.2.6. Demostración en lenguaje natural . . . 212
12.2.7. Por qué este puente importa . . . 212
12.3. Las sucesiones monótonas y acotadas son de Cauchy . . . 213
12.3.1. Abstracción y generalidad . . . 214
12.3.2. Nuevas herramientas . . . 215
12.3.2.1. Definición: monótona . . . 215
12.3.2.2. Teorema: monotonía del sucesor . . . 215
12.3.2.3. Táctica: `push_neg` . . . 215
12.3.3. El enunciado matemático . . . 215
12.3.4. Estrategia de la demostración . . . 216
12.3.5. Lema auxiliar pospuesto . . . 217
12.3.6. Demostración en Lean . . . 218
12.3.7. Demostración en lenguaje natural . . . 219
12.4. Brechas iteradas: el lema auxiliar . . . 220
12.4.1. El enunciado matemático . . . 221
12.4.2. Estrategia de la demostración . . . 221
12.4.3. Demostración en Lean . . . 221
12.4.4. Demostración en lenguaje natural . . . 222

12.4.5. Por qué este lema técnico importa . . . 223

13. Subsucesión monótona 225

13.1. Introducción: El teorema de la subsucesión monótona . . . 225
13.2. Nuevas definiciones . . . 226
13.2.1. Picos en una sucesión . . . 226
13.2.2. Picos no acotados . . . 226
13.2.3. Expresiones condicionales: `if-then-else` . . . 227
13.3. Nuevo teorema: `lt_of_not_ge` . . . 227
13.4. Teorema principal: Subsucesión monótona a partir de picos acotados . . . 228
13.4.1. Visión estratégica de la demostración . . . 228
13.4.2. Demostración en Lean . . . 230
13.4.3. Demostración en lenguaje natural . . . 231
13.5. El panorama general . . . 233
13.5.1. El papel de la construcción `if-then-else` . . . 234
13.5.2. Por qué funcionan las órbitas . . . 234
13.6. Perspectivas futuras . . . 235

14. El teorema de Bolzano-Weierstrass 236

14.1. Introducción: una piedra angular del análisis . . . 236
14.2. Nuevos teoremas . . . 237
14.2.1. `abs_le`: El complemento de `abs_lt` . . . 237
14.2.2. `IsCauchyOfAntitoneBdd`: El dual de nuestro resultado anterior . . . 238
14.2.3. `AntitoneSubseq_of_UnBddPeaks`: La mitad faltante . . . 239
14.3. El teorema principal: Bolzano-Weierstrass . . . 240
14.3.1. Visión estratégica . . . 240
14.3.2. Comprendiendo la acotación . . . 241
14.3.3. Demostración en Lean . . . 242
14.3.4. Demostración en lenguaje natural . . . 243
14.4. La arquitectura de la demostración . . . 244
14.4.1. La cimentación: sucesiones monótonas . . . 245
14.4.2. La dicotomía: picos y valles . . . 245
14.4.3. La extracción: de la dicotomía a la monotonía . . . 246
14.4.4. La síntesis: Bolzano-Weierstrass . . . 246

14.5. Por qué esto importa . . . 247
14.5.1. Compacidad . . . 247
14.5.2. Pruebas de existencia . . . 247
14.5.3. Aproximación . . . 248
14.5.4. Análisis funcional . . . 248
14.6. Contexto histórico . . . 248
14.7. Extensiones y variantes . . . 249
14.7.1. Dimensiones superiores . . . 249
14.7.2. Espacios métricos . . . 250
14.7.3. Convergencia débil . . . 250
14.7.4. Espacios topológicos . . . 250
14.8. Técnicas de demostración: Lo que aprendimos . . . 251
14.8.1. Análisis por casos con `by_cases` . . . 251
14.8.2. Descomposición de valores absolutos . . . 251
14.8.3. Argumentos de dualidad . . . 251
14.8.4. Aprovechamiento de resultados previos . . . 252
14.9. Mirando hacia adelante . . . 252

15. Los números reales **254**

16. Completitud de los números reales **269**
16.1. Introducción: La Propiedad de Completitud . . . 269
16.2. La Pregunta: ¿Son Completos los Reales? . . . 270
16.2.1. El Cuadro del Arreglo Doble . . . 270
16.2.2. El Objetivo . . . 270
16.3. La Idea Clave: La Diagonalización de Cantor . . . 271
16.3.1. El Intento Ingenuo . . . 271
16.3.2. La Solución: Elegir Puntos de Convergencia . . . 271
16.4. La Estrategia de Demostración . . . 271
16.4.1. Paso 1: Elegir los Índices . . . 272
16.4.2. Paso 2: Demostrar que y es de Cauchy . . . 272
16.4.3. Paso 3: Demostrar que $x_n \to$ `Real_of_CauSeq` hy . . . 273
16.5. La Demostración en Lean . . . 273
16.6. Comprendiendo la Demostración . . . 275

16.6.1. La Construcción de y 275
16.6.2. Por qué y es de Cauchy 275
16.6.3. Por qué $x_n \to L$ 276
16.7. La Importancia de la Completitud 276
16.7.1. Clausura Bajo Límites 276
16.7.2. Teoremas de Punto Fijo 276
16.7.3. Convergencia de Series 276
16.7.4. Construcción de Funciones 276
16.8. El Proceso de Completación 277
16.8.1. El Patrón General 277
16.8.2. Propiedades Clave de la Completación 277
16.8.3. Ejemplos en Matemáticas 277
16.9. El Argumento Diagonal 278
16.10.Reflexiones Filosóficas 278
16.10.1. El Precio de la Completitud 278
16.10.2. El Beneficio de la Completitud 278

Capítulo 10

Álgebra de límites y propiedades de orden

10.1. Producto de sucesiones

Habiendo establecido el teorema del límite algebraico para sumas y múltiplos escalares, ahora abordamos el producto de sucesiones convergentes. Este resultado, combinado con el trabajo anterior, completa la estructura algebraica de los límites: podemos sumar, multiplicar por constantes, multiplicar sucesiones entre sí e incluso dividir (siempre que el límite sea no nulo).

La estrategia de demostración refleja la regla del producto del cálculo, donde $(fg)' = f'g + g'f$. Introducimos artificialmente términos que nos permiten separar las contribuciones de cada sucesión.

10.1.1. El enunciado matemático

Teorema 10.1.1 (ProdLimNeNe). *Si $a, b : \mathbb{N} \to \mathbb{R}$ son sucesiones con $a(n) \to L$ y $b(n) \to M$, donde $L \neq 0$ y $M \neq 0$, y $c(n) = a(n) \cdot b(n)$ para todo n, entonces:*

$$c(n) \to L \cdot M$$

10.1.2. Enfoque estratégico

La idea clave es escribir:

$$a(n) \cdot b(n) - L \cdot M = \big(a(n) - L\big) \cdot b(n) + L \cdot \big(b(n) - M\big)$$

Esta descomposición separa el error en dos partes:

- $\big(a(n) - L\big) \cdot b(n)$: el error en a multiplicado por el valor de b.

- $L \cdot \big(b(n) - M\big)$: el límite L multiplicado por el error en b.

Para acotar el primer término usamos que b es convergente y por tanto acotada (por algún $K > 0$). Entonces $|a(n) - L|$ puede hacerse menor que $\varepsilon/(2K)$, lo que nos da control sobre ese término.

Para el segundo término usamos directamente que $b(n) \to M$, haciendo $|b(n) - M|$ menor que $\varepsilon/(2|L|)$.

10.1.3. Demostración en Lean

```
-- Demostración en lenguaje natural
-- ================================

-- Sea ε > 0. Como aₙ converge a L, existe N₁ tal que si n ≥ N₁,
-- entonces
--    |aₙ - L| < 1
-- Por tanto, para n ≥ N₁
--    |aₙ| = |aₙ - L + L|
--         ≤ |aₙ - L| + |L|
--         < 1 + |L|
-- Definamos
--    C = 1 + |L|
-- Entonces
--    C > 0
-- y, para n ≥ N₁
--    |aₙ| ≤ C
-- Ahora, como aₙ converge a L, existe N₂ tal que si n ≥ N₂,
-- entonces
--    |aₙ - L| < ε/(2(|M| + 1))
-- Por otra parte, como bₙ converge a M, existe N₃ tal que si
-- n ≥ N₃, entonces
--    |bₙ - M| < ε/(2C)
-- Sea
--    N = máx{N₁,N₂,N₃}
-- Si n ≥ N, entonces se cumplen las tres desigualdades
-- anteriores. Ahora estimamos:
--    |aₙbₙ - LM|
-- Sumamos y restamos aₙM:
--    aₙbₙ - LM = aₙbₙ - aₙM + aₙM - LM.
-- Entonces
--    |aₙbₙ - LM| = |aₙ(bₙ - M) + M(aₙ - L)|
-- Por la desigualdad triangular,
--    |aₙbₙ - LM| ≤ |aₙ||bₙ - M| +|M||aₙ - L|
-- Como n ≥ N₁,tenemos
--    |aₙ| ≤ C
-- Luego,
--    |aₙbₙ - LM| ≤ C|bₙ - M| + |M||aₙ - L|
```

```
-- Usando las cotas elegidas,
--    C|bₙ - M| < C•ε/(2C)
--              = ε/2
-- y
--    |M||aₙ - L| < |M|•ε/(2(|M| + 1))
-- Como
--    |M|/(|M| + 1) < 1
-- se sigue que
--    |M|•ε/(2(|M| + 1)) < ε/2
-- Por tanto,
--    |aₙbₙ - LM| < ε/2 + ε/2
--                = ε
-- Así, para todo ε > 0 existe N tal que si n ≥ N, entonces
--    |aₙbₙ - LM| < ε
-- Esto prueba que, para todo ε > 0, existe N tal que |aₙbₙ - LM| < ε
-- siempre que n ≥ N, es decir, aₙbₙ converge a LM.

-- Demostraciones en Lean 4
-- ========================

import Mathlib.Data.Real.Basic
import Analisis_en_Lean.C2_El_calculo_de_pi
import Analisis_en_Lean.C9B_Acotacion_de_convergentes
import Mathlib.Tactic

variable {a b c : ℕ → ℝ}
variable {L M : ℝ}

-- 1ª solución
-- ===========

example
  (ha : LimSuc a L)
  (hb : LimSuc b M)
  (hc : ∀ n, c n = a n * b n)
  : LimSuc c (L * M) :=
by
  intro ε hε
  -- ε : ℝ
  -- hε : ε > 0
  -- ⊢ ∃ k, ∀ n ≥ k, |c n - L * M| < ε
  obtain ⟨N₁, hN₁⟩ := ha 1 (by positivity)
  -- N₁ : ℕ
  -- hN₁ : ∀ n ≥ N₁, |a n - L| < 1
  have h1 : ∀ n ≥ N₁, |a n| < 1 + |L| := by grind
  set C := 1 + |L|
  -- h1 : ∀ n ≥ N₁, |a n| < C
  have h2 : C > 0 := by grind
  obtain ⟨N₂, hN₂⟩ := ha (ε / (2 * (|M| + 1))) (by positivity)
```

```
  -- N₂ : ℕ
  -- hN₂ : ∀ n ≥ N₂, |a n - L| < ε / (2 * (|M| + 1))
  obtain ⟨N₃, hN₃⟩ := hb (ε / (2 * C)) (by positivity)
  -- N₃ : ℕ
  -- hN₃ : ∀ n ≥ N₃, |b n - M| < ε / (2 * C)
  set N := max N₁ (max N₂ N₃)
  use N
  -- ⊢ ∀ n ≥ N, |c n - L * M| < ε
  intro n hn
  -- n : ℕ
  -- hn : n ≥ N
  -- ⊢ |c n - L * M| < ε
  have h3 : n ≥ N₁ := by grind
  have h4 : n ≥ N₂ := by grind
  have h5 : n ≥ N₃ := by grind
  have h6 : |a n| < C := by grind
  have h7 : |b n - M| < ε / (2 * C) := by grind
  have h8 : |a n - L| < ε / (2 * (|M| + 1)) := by grind
  have h9 : |M| * (ε / (2 * (|M| + 1))) < ε / 2 := by
              field_simp
              -- ⊢ |M| < |M| + 1
              norm_num
  calc |c n - L * M|
       = |a n * b n - L * M|                              := by grind
     _ = |a n * (b n - M) + M * (a n - L)|                := by grind
     _ ≤ |a n * (b n - M)| + |M * (a n - L)|              := by grind
     _ = |a n| * |b n - M| + |M| * |a n - L|              := by grind
     _ ≤ C * |b n - M| + |M| * |a n - L|                  := by gcongr
     _ < C * (ε / (2 * C)) + |M| * |a n - L|              := by gcongr
     _ ≤ C * (ε / (2 * C)) + |M| * (ε / (2 * (|M| + 1))) := by gcongr
     _ = ε / 2 + |M| * (ε / (2 * (|M| + 1)))              := by grind
     _ < ε / 2 + ε / 2                                    := by grind
     _ = ε                                                := by grind

-- 2ª solución
-- ===========

lemma L1
  (h : LimSuc a L)
  : ∃ k, ∀ n ≥ k, |a n| < 1 + |L| :=
by
  obtain ⟨k, hk⟩ := h 1 one_pos
  -- k : ℕ
  -- hk : ∀ n ≥ k, |a n - L| < 1
  use k
  -- ⊢ ∀ n ≥ k, |a n| < |L| + 1
  intro n hn
  -- n : ℕ
  -- hn : n ≥ k
```

```
  -- ⊢ |a n| < |L| + 1
  calc |a n|
      = |(a n - L) + L| := congrArg abs (sub_add_cancel (a n) L).symm
    _ ≤ |a n - L| + |L| := abs_add_le (a n - L) L
    _ < 1 + |L|         := (add_lt_add_iff_right |L|).mpr (hk n hn)

example
  (ha : LimSuc a L)
  (hb : LimSuc b M)
  (hc : ∀ n, c n = a n * b n)
  : LimSuc c (L * M) :=
by
  intro ε hε
  -- ε : ℝ
  -- hε : ε > 0
  -- ⊢ ∃ k, ∀ n ≥ k, |c n - L * M| < ε
  obtain ⟨N₁, hN₁⟩ := L1 ha
  -- N₁ : ℕ
  -- hN₁ : ∀ n ≥ N₁, |a n| < 1 + |L|
  set C := 1 + |L| with hC
  -- hC : C = 1 + |L|
  have h2 : C > 0 := by positivity
  obtain ⟨N₂, hN₂⟩ := ha (ε / (2 * (|M| + 1))) (by positivity)
  -- N₂ : ℕ
  -- hN₂ : ∀ n ≥ N₂, |a n - M| < ε / (2 * (|M| + 1))
  obtain ⟨N₃, hN₃⟩ := hb (ε / (2 * C)) (by positivity)
  -- N₃ : ℕ
  -- hN₃ : ∀ n ≥ N₃, |b n - M| < ε / (2 * C)
  set N := max N₁ (max N₂ N₃)
  use N
  -- ⊢ ∀ n ≥ N, |c n - L * M| < ε
  intro n hn
  -- n : ℕ
  -- hn : n ≥ N
  -- ⊢ |c n - L * M| < ε
  have h3 : n ≥ N₁ := by omega
  have h4 : n ≥ N₂ := by omega
  have h5 : n ≥ N₃ := by omega
  have h6 : |a n| < C := by
    calc |a n|
        < 1 + |L| := hN₁ n h3
      _ = C       := rfl
  have h7 : |b n - M| < ε / (2 * C) := hN₃ n h5
  have h8 : |a n - L| < ε / (2 * (|M| + 1)) := hN₂ n h4
  have h9 : |M| * (ε / (2 * (|M| + 1))) < ε / 2 := by
              field_simp
              -- ⊢ |M| < |M| + 1
              norm_num
  calc |c n - L * M|
```

```
    = |a n * b n - L * M| :=
        congrArg (|· - L * M|) (hc n)
  _ = |a n * (b n - M) + M * (a n - L)| :=
        by ring_nf
  _ ≤ |a n * (b n - M)| + |M * (a n - L)| :=
        abs_add_le (a n * (b n - M)) (M * (a n - L))
  _ = |a n| * |b n - M| + |M| * |a n - L| :=
        by simp only [abs_mul]
  _ ≤ C * |b n - M| + |M| * |a n - L| :=
        by gcongr
  _ < C * (ε / (2 * C)) + |M| * |a n - L| :=
        by gcongr
  _ ≤ C * (ε / (2 * C)) + |M| * (ε / (2 * (|M| + 1))) :=
        by gcongr
  _ = ε / 2 + |M| * (ε / (2 * (|M| + 1))) :=
        by field_simp
  _ < ε / 2 + ε / 2 :=
        by gcongr
  _ = ε :=
        add_halves ε
```

10.1.4. Demostración en lenguaje natural

Sea $\varepsilon > 0$ dado. Como $b(n) \to M$ con $M \neq 0$, por `BddOfConvNonzero` existe $K > 0$ tal que $|b(n)| \leq K$ para todo n.

Definamos:

$$\varepsilon_1 := \frac{\varepsilon}{2K}, \qquad \varepsilon_2 := \frac{\varepsilon}{2|L|}$$

Nótese que $\varepsilon_1 > 0$ (pues $K > 0$) y $\varepsilon_2 > 0$ (pues $L \neq 0$ implica $|L| > 0$).

Como $a(n) \to L$, existe N_1 tal que para todo $n \geq N_1$:

$$|a(n) - L| < \varepsilon_1 = \frac{\varepsilon}{2K}$$

Como $b(n) \to M$, existe N_2 tal que para todo $n \geq N_2$:

$$|b(n) - M| < \varepsilon_2 = \frac{\varepsilon}{2|L|}$$

Sea $N := N_1 + N_2$. Para cualquier $n \geq N$ tenemos $n \geq N_1$ y $n \geq N_2$.

Usando la identidad algebraica $a(n) \cdot b(n) - L \cdot M = (a(n) - L) \cdot b(n) + L \cdot (b(n) - M)$ y aplicando la desigualdad triangular:

$$\begin{aligned} |c(n) - L \cdot M| &= |(a(n) - L) \cdot b(n) + L \cdot (b(n) - M)| \\ &\leq |(a(n) - L) \cdot b(n)| + |L \cdot (b(n) - M)| \\ &= |a(n) - L| \cdot |b(n)| + |L| \cdot |b(n) - M| \end{aligned}$$

Para el primer término, como $|b(n)| \leq K$ y $|a(n) - L| < \varepsilon/(2K)$:

$$|a(n) - L| \cdot |b(n)| < \frac{\varepsilon}{2K} \cdot K = \frac{\varepsilon}{2}$$

Para el segundo término, como $|b(n) - M| < \varepsilon/(2|L|)$:

$$|L| \cdot |b(n) - M| < |L| \cdot \frac{\varepsilon}{2|L|} = \frac{\varepsilon}{2}$$

Por tanto:

$$|c(n) - L \cdot M| < \frac{\varepsilon}{2} + \frac{\varepsilon}{2} = \varepsilon$$

□

10.1.5. Conexión con el cálculo: ¿por qué sumar y restar $b(n) \cdot L$?

La técnica de la demostración refleja la regla del producto del cálculo: $(fg)' = f'g + g'f$. Para entender por qué sumamos y restamos $b(n) \cdot L$, pensemos geométricamente en el área de un rectángulo (al fin y al cabo, la multiplicación es rectángulos).

Cuando $a(n)$ y $b(n)$ están cerca de sus límites L y M, el producto $a(n) \cdot b(n)$ representa el área de un rectángulo de lados $a(n)$ y $b(n)$, mientras que $L \cdot M$ es el área del rectángulo límite de lados L y M.

La diferencia de áreas satisface:

$$a(n) \cdot b(n) - L \cdot M = L \cdot (b(n) - M) + M \cdot (a(n) - L) + (a(n) - L) \cdot (b(n) - M)$$

La clave es reemplazar M por $b(n)$ en el término central (sumando y restando $b(n) \cdot L$) para obtener:

$$a(n) \cdot b(n) - L \cdot M = (a(n) - L) \cdot b(n) + L \cdot (b(n) - M)$$

¿Por qué es mejor esto? Porque ahora cada término aísla una fuente de error:

- $(a(n) - L) \cdot b(n)$: error pequeño en a por la cantidad acotada $b(n)$.
- $L \cdot (b(n) - M)$: la constante fija L por el error pequeño en b.

10.1.6. Completando el teorema del límite algebraico

Este resultado completa el teorema del límite algebraico. Combinado con los resultados anteriores sobre sumas, múltiplos escalares y recíprocos de sucesiones con límite no nulo, podemos calcular el límite de cualquier combinación algebraica de sucesiones convergentes.

Por ejemplo, si $a(n) \to L$ y $b(n) \to M$, el límite de

$$\frac{a(n)^2 + 2a(n) + b(n)}{3b(n) + 2 - a(n)^2}$$

es, siempre que el límite del denominador sea no nulo:

$$\frac{L^2 + 2L + M}{3M + 2 - L^2}$$

10.2. Paso al límite en desigualdades

El paso al límite en desigualdades establece una relación fundamental entre desigualdades y límites: si una sucesión está acotada superiormente (o inferiormente) por una constante, entonces su límite también lo está. Este resultado respeta la estructura de orden de los números reales.

10.2.1. Nuevas herramientas

Definición 10.2.1 (SeqBddBy)**.** Una sucesión $a : \mathbb{N} \to \mathbb{R}$ está *acotada por* M (denotado SeqBddBy(a, M)) si $a(n) \leq M$ para todo $n \in \mathbb{N}$:

$$\text{SeqBddBy}(a, M) \iff \forall n \in \mathbb{N},\ a(n) \leq M$$

Nótese la diferencia entre SeqBdd (acotada: $|a(n)| \leq M$) y SeqBddBy (acotada superiormente: $a(n) \leq M$).

10.2.2. El enunciado matemático

Teorema 10.2.2 (OrderLimLe)**.** *Si $a : \mathbb{N} \to \mathbb{R}$ converge a L y $a(n) \leq K$ para todo n, entonces $L \leq K$.*

10.2.3. Enfoque estratégico

Lo demostramos por contradicción. Supongamos $L > K$. Entonces $L - K > 0$, y podemos usar esta cantidad positiva como ε en la definición de convergencia.

Por convergencia, existe N tal que para todo $n \geq N$:

$$|a(n) - L| < L - K$$

Esto implica $a(n) > K$, lo que contradice la hipótesis $a(n) \leq K$.

10.2.4. Demostración en Lean

```
-- Demostración en lenguaje natural
-- ================================

-- Lo demostramos por contradicción. Supongamos que
--    L > M                                                    (1)
-- y llegaremos a la contradicción M < M.
--
-- De (1), se tiene
--    L - M > 0
```

```
-- y, usando esta cantidad en la convergencia de aₙ, se obtiene
-- un k tal que
--    ∀ n ≥ k, |aₙ - L| < L - M                                (2)
-- En particular, para n = k (que satisface k ≥ k), de (2) se
-- obtiene
--    |aₖ - L| < L - M                                         (3)
-- Además,
--    L - aₖ ≤ |aₖ - L|                                        (4)
-- ya que
--    L - aₖ = -(aₖ - L)
--           ≤ |aₖ - L|
-- Finalmente,
--    M = L - (L - M)
--      < L - |aₖ - L|  [por (3)]
--      ≤ L - (L - aₖ)  [por (4)]
--      = aₖ
--      ≤ M             [porque M es cota superior de aₙ]
-- Por tanto, M < M.

-- Demostraciones en Lean 4
-- ========================

import Mathlib.Data.Real.Basic
import Analisis_en_Lean.C2_El_calculo_de_pi
import Mathlib.Tactic

def CotaSup (a : ℕ → ℝ) (M : ℝ) : Prop :=
  ∀ n, a n ≤ M

variable {a : ℕ → ℝ}
variable {L M : ℝ}

-- 1ª solución
-- ===========

example
  (ha : LimSuc a L)
  (hM : CotaSup a M)
  : L ≤ M :=
by
  by_contra hL
  -- hL : ¬L ≤ M
  -- ⊢ False
  apply lt_irrefl M
  -- ⊢ M < M
  obtain ⟨k, hk⟩ := ha (L - M) (by grind)
  -- k : ℕ
  -- hk : ∀ n ≥ k, |a n - L| < L - M
  calc M
```

```
      < a k := by grind
    _ ≤ M   := hM k

-- 2ª solución
-- ===========

example
  (ha : LimSuc a L)
  (hM : CotaSup a M)
  : L ≤ M :=
by
  by_contra hL
  -- hL : ¬L ≤ M
  -- ⊢ False
  apply lt_irrefl M
  -- ⊢ M < M
  have h1 : L > M := not_le.mp hL
  have h2 : L - M > 0 := sub_pos.mpr h1
  obtain ⟨k, hk⟩ := ha (L - M) h2
  -- k : ℕ
  -- hk : ∀ n ≥ k, |a n - L| < L - M
  calc M
      = L - (L - M)    := by grind
    _ < L - |a k - L|  := by grind
    _ ≤ L - (L - a k)  := by grind
    _ = a k            := by grind
    _ ≤ M              := hM k

-- 3ª solución
-- ===========

example
  (ha : LimSuc a L)
  (hM : CotaSup a M)
  : L ≤ M :=
by
  by_contra hL
  -- hL : ¬L ≤ M
  -- ⊢ False
  apply lt_irrefl M
  -- ⊢ M < M
  have h1 : L - M > 0 := sub_pos.mpr (not_le.mp hL)
  obtain ⟨k, hk⟩ := ha (L - M) h1
  -- k : ℕ
  -- hk : ∀ n ≥ k, |a n - L| < L - M
  have h2 : |a k - L| < L - M := hk k (le_refl k)
  have h3 : L - a k ≤ |a k - L| := by
    calc L - a k
        = -(a k - L) := by ring
```

```
      _ ≤ |a k - L|  := neg_le_abs (a k - L)
  calc M
       = L - (L - M)    := by ring
    _ < L - |a k - L|   := sub_lt_sub_left h2 L
    _ ≤ L - (L - a k)   := by gcongr
    _ = a k             := by ring
    _ ≤ M               := hM k

-- 4ª solución
-- ===========

example
  (ha : LimSuc a L)
  (hM : CotaSup a M)
  : L ≤ M :=
by
  by_contra hL
  -- hL : ¬L ≤ M
  -- ⊢ False
  apply lt_irrefl M
  -- ⊢ M < M
  obtain ⟨k, hk⟩ := ha (L - M) (sub_pos.mpr (not_le.mp hL))
  -- k : ℕ
  -- hk : ∀ n ≥ k, |a n - L| < L - M
  have h1 : |a k - L| < L - M := hk k (le_refl k)
  have h2 : L - a k ≤ |a k - L| := by
    calc L - a k
         = -(a k - L) := (neg_sub (a k) L).symm
      _ ≤ |a k - L|  := neg_le_abs (a k - L)
  calc M
       = L - (L - M)    := (sub_sub_self L M).symm
    _ < L - |a k - L|   := sub_lt_sub_left h1 L
    _ ≤ L - (L - a k)   := sub_le_sub_left h2 L
    _ = a k             := sub_sub_self L (a k)
    _ ≤ M               := hM k

-- Solución del libro
-- ==================

example
  (ha : LimSuc a L)
  (hM : CotaSup a M)
  : L ≤ M :=
by
  by_contra hL
  -- hL : ¬L ≤ M
  -- ⊢ False
  have hL' : M < L := by bound
  have hLM : 0 < (L - M) := by linarith [hL']
```

```
  obtain ⟨k, hk⟩ := ha (L - M) hLM
  -- k : ℕ
  -- hk : ∀ n ≥ k, |a n - L| < L - M
  specialize hk k (by bound)
  -- hk : |a k - L| < L - M
  rewrite [abs_lt] at hk
  -- hk : -(L - M) < a k - L ∧ a k - L < L - M
  have f1 : L - (L - M) < a k := by linarith [hk.1]
  have f2 : M = L - (L - M) := by ring_nf
  specialize hM k
  -- hM : a k ≤ M
  linarith [f2, hM, f1]

-- Lemas usados
-- ============

variable (x y : ℝ)
#check (le_refl x : x ≤ x)
#check (lt_irrefl x : ¬x < x)
#check (neg_le_abs x : -x ≤ |x|)
#check (neg_sub x y : -(x - y) = y - x)
#check (not_le : ¬x ≤ y ↔ y < x)
#check (sub_le_sub_left : x ≤ y → ∀ z, z - y ≤ z - x)
#check (sub_lt_sub_left : x < y → ∀ z, z - y < z - x)
#check (sub_pos : 0 < x - y ↔ y < x)
#check (sub_sub_self x y : x - (x - y) = y)
```

10.2.5. Demostración en lenguaje natural

Procedemos por contradicción. Supongamos $L > K$.

Entonces $L - K > 0$. Como $a(n) \to L$, por la definición de convergencia con $\varepsilon = L - K > 0$, existe N tal que para todo $n \geq N$:

$$|a(n) - L| < L - K$$

En particular, tomando $n = N$:

$$|a(N) - L| < L - K \implies -(L - K) < a(N) - L < L - K$$

De la desigualdad izquierda:

$$a(N) > L - (L - K) = K$$

Pero por hipótesis $a(N) \leq K$, lo que es una contradicción.

Por tanto, $L > K$ es falso, y concluimos $L \leq K$. □

10.2.6. Importancia de la preservación del orden

Este teorema establece que los límites preservan las desigualdades débiles. Nótese que las desigualdades estrictas no se preservan necesariamente: si $a(n) < K$ para todo n, solo podemos concluir $L \leq K$, no $L < K$.

Por ejemplo, la sucesión $a(n) = 1/n$ satisface $a(n) > 0$ para todo n, pero $\lim_{n\to\infty} a(n) = 0$, que no es estrictamente positivo.

El paso al límite en desigualdades se usa para demostrar:

- El teorema del emparedado.
- El teorema de convergencia monótona.
- Diversos criterios de comparación para series.
- Propiedades del supremo e ínfimo de sucesiones.

10.3. Subsucesiones

Una subsucesión se forma seleccionando términos de una sucesión preservando su orden. Hay dos perspectivas equivalentes. La mayoría de los autores llaman $a_{\sigma(n)}$ «subsucesión de a», viéndola como una nueva sucesión extraída de la original. Sin embargo, preferimos llamar σ la subsucesión: es la función estrictamente creciente $\sigma : \mathbb{N} \to \mathbb{N}$ que indica qué términos conservar. La subsucesión de a determinada por σ es entonces la composición $a \circ \sigma$.

Geométricamente, imagine la gráfica de la sucesión original $a(n)$. Para formar una subsucesión se eliminan algunos términos y se reindexan los restantes. Si conservamos solo los términos en las posiciones $0, 2, 5, 7, 11, \ldots$, los reetiquetamos como $0, 1, 2, 3, 4, \ldots$ La función σ codifica este reetiquetado: $\sigma(0) = 0$, $\sigma(1) = 2$, $\sigma(2) = 5$, etc.

El resultado fundamental es que si una sucesión converge, toda subsucesión converge al mismo límite.

10.3.1. Nuevas herramientas

Definición 10.3.1 (Subseq)**.** Una función $\sigma : \mathbb{N} \to \mathbb{N}$ es una *subsucesión* (denotado $\text{Subseq}(\sigma)$) si es estrictamente creciente:

$$\text{Subseq}(\sigma) \iff \forall\, i, j \in \mathbb{N},\ i < j \implies \sigma(i) < \sigma(j)$$

La subsucesión de a determinada por σ es la composición $a \circ \sigma : \mathbb{N} \to \mathbb{R}$, donde $(a \circ \sigma)(n) = a(\sigma(n))$.

Lema 10.3.2 (SubseqGe)**.** *Si σ es una subsucesión, entonces $n \leq \sigma(n)$ para todo n. Esto puede demostrarse por inducción y captura la intuición de que estamos «expandiendo» los índices.*

10.3.2. El enunciado matemático

Teorema 10.3.3 (SubseqConv). *Si $a : \mathbb{N} \to \mathbb{R}$ converge a L y $\sigma : \mathbb{N} \to \mathbb{N}$ es una subsucesión, entonces $a \circ \sigma$ también converge a L.*

10.3.3. Enfoque estratégico

La demostración es notablemente sencilla. Dado $\varepsilon > 0$, usamos la convergencia de a para encontrar N tal que $|a(m) - L| < \varepsilon$ para todo $m \geq N$.

Para cualquier $n \geq N$, necesitamos $|(a \circ \sigma)(n) - L| < \varepsilon$. La observación clave es que $\sigma(n) \geq n \geq N$ (usando el lema $n \leq \sigma(n)$), de modo que $\sigma(n) \geq N$ y por tanto $|a(\sigma(n)) - L| < \varepsilon$.

10.3.4. Demostración en Lean

```
-- ---------------------------------------------------------------
-- Una subsucesión se obtiene aplicando a la sucesión original una
-- función de extracción; es decir, una función φ : ℕ → ℕ
-- estrictamente creciente. Por ejemplo, la subsucesión
--    u₀, u₂, u₄, u₆, ...
-- se obtiene con la función de extracción φ definida por
-- φ(n) = 2n.
--
-- Las definiciones anteriores se formalizan en Lean 4 como:
--
--    -- φ es una función de extracción.
--    def extraccion (φ : ℕ → ℕ) :=
--      StrictMono φ
--
--    -- v es una subsucesión de u.
--    def subsucesion (v u : ℕ → ℝ) :=
--      ∃ φ, extraccion φ ∧ v = u ∘ φ
--
--    -- a es el límite de u.
--    def LimSuc (u : ℕ → ℝ) (a : ℝ) :=
--      ∀ ε > 0, ∃ k : ℕ, ∀ n ≥ k, |u n - a| < ε
--
-- Demostrar que toda subsucesión de una sucesión convergente
-- converge al mismo límite que la sucesión.
-- ---------------------------------------------------------------


-- Demostración en lenguaje natural
-- ================================

-- Usaremos el siguiente lema: Si φ es una función de extracción,
```

```
-- entonces
--    ∀ n, n ≤ φ(n)
--
-- Por ser v una subsucesión de u, existe una función de
-- extracción φ tal que
--    v = u ∘ φ                                                (3)
--
-- Tenemos que demostrar que para cada ε > 0, existe un k ∈ ℕ tal
-- que
--     ∀ n ≥ k, |v(n) - a| < ε                                 (1)
--
-- Puesto que a es el límite de u, , existe un k ∈ ℕ tal
-- que
--     ∀ n ≥ k, |u(n) - a| < ε                                 (2)
--
-- Veamos que k verifica (1). Para ello, sea n ≥ k. Entonces,
--    φ(n) ≥ k                                                 (4)
-- ya que
--    φ(n) ≥ n    [por el Lema]
--         ≥ k
-- Luego,
--    |v(n) - a| = |(u ∘ φ )(n) - a|    [por (3)]
--               = |u(φ(n)) - a|
--               < ε                   [por (2) y (4)]

-- Demostraciones con Lean4
-- ========================

import Mathlib.Data.Real.Basic
import Analisis_en_Lean.C2_El_calculo_de_pi

open Nat

variable {u v : ℕ → ℝ}
variable {a : ℝ}
variable {φ : ℕ → ℕ}

-- φ es una función de extracción.
def extraccion (φ : ℕ → ℕ):=
  StrictMono φ

-- v es una subsucesión de u.
def subsucesion (v u : ℕ → ℝ) :=
  ∃ φ, extraccion φ ∧ v = u ∘ φ

-- 1ª demostración
-- ===============

example
```

```
  (hv : subsucesion v u)
  (ha : LimSuc u a)
  : LimSuc v a :=
by
  obtain ⟨φ, hφ, hφ'⟩ := hv
  -- φ : ℕ → ℕ
  -- hφ : extraccion φ
  -- hφ' : v = u ∘ φ
  intros ε hε
  -- ε : ℝ
  -- hε : ε > 0
  -- ⊢ ∃ k, ∀ n ≥ k, |v n - a| < ε
  obtain ⟨k, hk⟩ := ha ε hε
  -- k : ℕ
  -- hk : ∀ n ≥ k, |u n - a| < ε
  use k
  -- ⊢ ∀ n ≥ k, |v n - a| < ε
  intros n hn
  -- n : ℕ
  -- hn : n ≥ k
  -- ⊢ |v n - a| < ε
  have h1 : φ n ≥ k :=
    calc φ n
         ≥ n := StrictMono.le_apply hφ
       _ ≥ k := hn
  calc |v n - a|
       = |(u ∘ φ ) n  - a| := by rw [hφ']
     _ = |u (φ n) - a|     := rfl
     _ < ε                 := hk (φ n) h1

-- 2ª demostración
-- ===============

example
  (hv : subsucesion v u)
  (ha : LimSuc u a)
  : LimSuc v a :=
by
  obtain ⟨φ, hφ, rfl⟩ := hv
  -- φ : ℕ → ℕ
  -- hφ : extraccion φ
  -- ⊢ LimSuc (u ∘ φ) a
  intros ε hε
  -- ε : ℝ
  -- hε : ε > 0
  -- ⊢ ∃ k, ∀ n ≥ k, |(u ∘ φ) n - a| < ε
  obtain ⟨k, hk⟩ := ha ε hε
  -- k : ℕ
  -- hk : ∀ n ≥ k, |u n - a| < ε
```

```
  use k
  -- ⊢ ∀ n ≥ k, |(u ∘ φ) n - a| < ε
  intros n hn
  -- n : ℕ
  -- hn : n ≥ k
  -- ⊢ |(u ∘ φ) n - a| < ε
  have h1 : φ n ≥ k :=
    calc φ n
         ≥ n := hφ.le_apply
       _ ≥ k := hn
  calc |(u ∘ φ) n  - a|
     _ = |u (φ n) - a|     := rfl
     _ < ε                 := hk (φ n) h1

-- 3ª demostración
-- ===============

example
  (hv : subsucesion v u)
  (ha : LimSuc u a)
  : LimSuc v a :=
by
  obtain ⟨φ, hφ, rfl⟩ := hv
  -- φ : ℕ → ℕ
  -- hφ : extraccion φ
  -- ⊢ LimSuc (u ∘ φ) a
  intros ε hε
  -- ε : ℝ
  -- hε : ε > 0
  -- ⊢ ∃ k, ∀ n ≥ k, |(u ∘ φ) n - a| < ε
  obtain ⟨k, hk⟩ := ha ε hε
  -- k : ℕ
  -- hk : ∀ n ≥ k, |u n - a| < ε
  use k
  -- ⊢ ∀ n ≥ k, |(u ∘ φ) n - a| < ε
  intros n hn
  -- n : ℕ
  -- hn : n ≥ k
  -- ⊢ |(u ∘ φ) n - a| < ε
  apply hk
  -- ⊢ φ n ≥ k
  apply le_trans hn
  -- ⊢ n ≤ φ n
  exact hφ.le_apply

-- 4ª demostración
-- ===============

example
```

```
  (hv : subsucesion v u)
  (ha : LimSuc u a)
  : LimSuc v a :=
by
  obtain ⟨φ, hφ, rfl⟩ := hv
  -- φ : ℕ → ℕ
  -- hφ : extraccion φ
  -- ⊢ LimSuc (u ∘ φ) a
  intros ε hε
  -- ε : ℝ
  -- hε : ε > 0
  -- ⊢ ∃ k, ∀ n ≥ k, |(u ∘ φ) n - a| < ε
  obtain ⟨k, hk⟩ := ha ε hε
  -- k : ℕ
  -- hk : ∀ n ≥ k, |u n - a| < ε
  exact ⟨k, fun n hn ↦ hk (φ n) (le_trans hn hφ.le_apply)⟩

-- Lemas usados
-- ============

variable (m n x y z : ℕ)
#check (Nat.zero_le n : 0 ≤ n)
#check (le_trans : x ≤ y → y ≤ z → x ≤ z)
```

10.3.5. Demostración en lenguaje natural

Sea $\varepsilon > 0$ dado. Como $a(n) \to L$, existe N tal que para todo $m \geq N$:

$$|a(m) - L| < \varepsilon$$

Afirmamos que este mismo N sirve para la subsucesión $a \circ \sigma$. Es decir, para todo $n \geq N$:

$$|(a \circ \sigma)(n) - L| < \varepsilon$$

Sea $n \geq N$. Por el lema `SubseqGe`, $n \leq \sigma(n)$, luego:

$$\sigma(n) \geq n \geq N$$

Como $\sigma(n) \geq N$:

$$|(a \circ \sigma)(n) - L| = |a(\sigma(n)) - L| < \varepsilon$$

□

10.3.6. Por qué importa

El teorema de la subsucesión tiene varias consecuencias importantes:

- Si una sucesión tiene dos subsucesiones que convergen a límites distintos, la sucesión no converge. Esto proporciona una herramienta poderosa para probar divergencia.
- En el teorema de Bolzano-Weierstrass, toda sucesión acotada tiene una subsucesión convergente.
- Las subsucesiones se usan para extraer comportamiento «interesante», como lím inf y lím sup.
- El concepto se generaliza a espacios métricos y topológicos, donde juega un papel crucial en los argumentos de compacidad.

10.4. Ejemplo de subsucesión: sucesiones oscilantes

Aplicamos el teorema de la subsucesión a un ejemplo concreto: la sucesión $a(n) = (-1)^n$, que oscila entre -1 y 1. Esta sucesión no converge, pero tiene subsucesiones convergentes.

Extrayendo los términos de índice par obtenemos una subsucesión constante que converge a 1. Análogamente, los términos de índice impar convergen a -1.

10.4.1. Nuevas herramientas

La táctica `let`. La táctica `let` es similar a `have`, pero crea un nuevo término o función con nombre en lugar de solo demostrar una proposición. Para crear una función en Lean se usa la palabra clave `fun`:

```
fun x ↦ expresion
```

lo que define una función que toma la entrada x y devuelve `expresión`. Por ejemplo, `fun x ↦ x^2` es la función cuadrática.

En el ejemplo de la subsucesión escribimos:

```
let σ : N → N := fun n ↦ 2 * n
```

que define $\sigma(n) = 2n$, seleccionando los términos de índice par.

10.4.2. El enunciado matemático

Teorema 10.4.1. *Si $a : \mathbb{N} \to \mathbb{R}$ está definida por $a(n) = (-1)^n$, entonces existe una subsucesión σ y un límite L tales que σ es estrictamente creciente y $a \circ \sigma$ converge a L.*

10.4.3. Enfoque estratégico

Definimos $\sigma(n) = 2n$, que selecciona los índices pares: $0, 2, 4, 6, \ldots$

Primero verificamos que σ es estrictamente creciente: si $i < j$, entonces $2i < 2j$.

Luego calculamos $(a \circ \sigma)(n) = a(2n) = (-1)^{2n} = 1$ para todo n.

Finalmente, esta sucesión constante converge trivialmente a 1: para cualquier $\varepsilon > 0$, $|1 - 1| = 0 < \varepsilon$.

10.4.4. Demostración en Lean

```
Statement (a : N → R) (ha : ∀ n, a n = (-1) ^ n) :
    ∃ σ L, Subseq σ ∧ SeqLim (a ∘ σ) L := by
  let σ : N → N := fun n ↦ 2 * n
  use σ, 1
  split_ands
  · intro i j hij
    change 2 * i < 2 * j
    linarith [hij]
  · intro ε hε
    use 0
    intro n hn
    change |a (2 * n) - 1| < ε
    specialize ha (2 * n)
    rewrite [ha]
    have f1 : (-1 : R) ^ (2 * n) = 1 := by bound
    rewrite [f1]
    norm_num
    apply hε
```

10.4.5. Demostración en lenguaje natural

Paso 1: σ es una subsucesión.

Sean $i, j \in \mathbb{N}$ con $i < j$. Entonces:

$$\sigma(i) = 2i < 2j = \sigma(j)$$

Por tanto σ es estrictamente creciente, y en consecuencia es una subsucesión.

Paso 2: $a \circ \sigma$ converge a 1.

Sea $\varepsilon > 0$ dado. Para todo $n \geq 0$:

$$\begin{aligned}(a \circ \sigma)(n) &= a(\sigma(n)) \\ &= a(2n) \\ &= (-1)^{2n} \\ &= \left((-1)^2\right)^n \\ &= 1^n \\ &= 1\end{aligned}$$

Por tanto:

$$\begin{aligned}|(a \circ \sigma)(n) - 1| &= |1 - 1| \\ &= 0 \\ &< \varepsilon\end{aligned}$$

□

10.4.6. Implicaciones para la divergencia

Este ejemplo demuestra que $a(n) = (-1)^n$ no converge. Podemos extraer subsucesiones que convergen a 1 (índices pares) y a -1 (índices impares). Como una sucesión convergente debe tener todas sus subsucesiones convergiendo al mismo límite, y hemos encontrado dos con límites distintos, la sucesión a no puede converger.

Esta técnica —mostrar la divergencia exhibiendo dos subsucesiones con límites distintos— es una herramienta poderosa en el análisis.